LabLog — Digital Practical Lab Book

SE5070 Enterprise Mobility · Assignment Report

MSc in IT (Enterprise Application Development) · 2026 · Year 1 · Semester 1

Student	Thanura Manjitha Peiris
Index number	MS26928210
Project area	0 — Edtech
Platform	Native iOS (Swift / SwiftUI) · custom Express + TypeScript API · Supabase (Postgres, Auth, Storage)
Source code	github.com/thxnura/lablog-ios · github.com/thxnura/lablog-backend
Submitted	October 2026

Contents — 1 Project area · 2 Idea and features · 3 Architecture and design · 4 Offline access · 5 Constraint design · 6 Native hardware · 7 UI/UX flow · 8 Custom components · 9 Decision records, failure logs and AI use · 10 Testing · 11 Limitations · Appendix

---

1. Project area from the index number

```
Index number : MS26928210
Digits       : 2 + 6 + 9 + 2 + 8 + 2 + 1 + 0 = 30
Area         : 30 mod 5 = 0  →  Edtech
```

LabLog is an Edtech application, which matches the assigned area.

---

2. Project idea and features

2.1 The problem

University practicals are still recorded in paper lab books or loose Word files. That causes three problems:

- **Lecturers can't verify the work.** A tick on paper doesn't show that a step was done, done in the right order, or done at all.
- **Procedures get skipped or reordered.** Students jump ahead (for example, titrating before adding the indicator) and the mistake only shows in the final result.
- **Lab connectivity is poor.** Labs are often in basements or shielded rooms, so a cloud-only app stops working exactly where it is needed.

2.2 The solution

LabLog is an **offline-first iPhone lab book**. A student opens a lab, picks an experiment, and works through it as a **constrained checklist**:

- A step stays locked until its prerequisites are done.

- Steps that need **photo evidence** can't be completed without a photo.
- Steps with a **waiting time** (dissolving, incubating, timing oscillations) can't be completed until the time has passed. The countdown follows the student into the **Dynamic Island** and Lock Screen.

The finished record is **stamped with the student's location** and submitted. Everything is saved on the phone first and **syncs to the backend when a connection returns**. A lecturer signs in to the same app and gets a **Review** tab. It shows every submission with step times, photos, the observation and the location, from a lecturer-only API.

2.3 Why it is worthwhile

- **It turns a paper form into verifiable evidence.** Step order is enforced, every completion is timestamped, photos are attached to the step they prove, and the GPS stamp shows where the record was submitted.
- **It teaches as well as records.** Every step has a guide (how-to sub-steps, a safety note, the expected result), so the app supports learning mid-experiment rather than only logging it.
- **Offline is essential here, not a bolt-on.** Labs are where connectivity is worst, so the app is designed around working without a network.
- **It generalises.** Any procedural practical (chemistry, physics, biology, engineering workshops, clinical skills) is a graph of steps with conditions, so new experiments are data, not code.

2.4 Feature list

Feature	What it does	Brief requirement
Lab and experiment catalog	Browse labs and experiments; works offline from first launch (bundled catalog, refreshed from the API)	Backend, offline
Constrained step checklist	Steps unlock by precedence; photo and timer guards; explains *why* a step is locked	Constraint design
Step guide	How-to sub-steps, safety note, expected result; jump to blocking steps	UX
Photo evidence	Camera capture per step, stored on the device, uploaded later	Native hardware (camera)
Location stamp	One-shot GPS fix on submit, shown on a map receipt	Native hardware (GPS)
Live timers	Countdown dial in the app, Dynamic Island and Lock Screen Live Activity, "ready" notification, *Mark done* from the island	Native platform features
Offline outbox and sync	Local-first storage; automatic sync on reconnect, with retry and backoff	Offline + sync
Review and submit	Summary of what will be submitted; records become read-only	UX
Records and sync status	All records with progress and sync state; connection, account and outbox status	UX
Lecturer review	A Review tab for lecturers: submissions per experiment, with steps and times, photos, observation, step notes and map; backed by a lecturer-only API	Backend, UX

Feature	What it does	Brief requirement
Editing a draft	Retake a step's photo (remove it until the step is done), add a note to any completed step, or discard the whole draft; every change syncs, deletes included	Offline + sync, constraint design
Server address	Sync → Server points the app at the Mac on any network (a phone hotspot at a demo), with a connection test; no rebuild	Backend, UX

2.5 What makes it different

Most lab-book apps are digitised forms. LabLog models an experiment as a **dependency graph with guard conditions**, and one pure algorithm decides what a student may do next. The same rule is applied everywhere: the checklist, the step guide, the Dynamic Island's *Mark done* button and the submit sheet. On the server, ownership rules stop one student's data being overwritten by another's. The result is a lab book that is **hard to fake and still works with no signal**.

3. Architecture and design

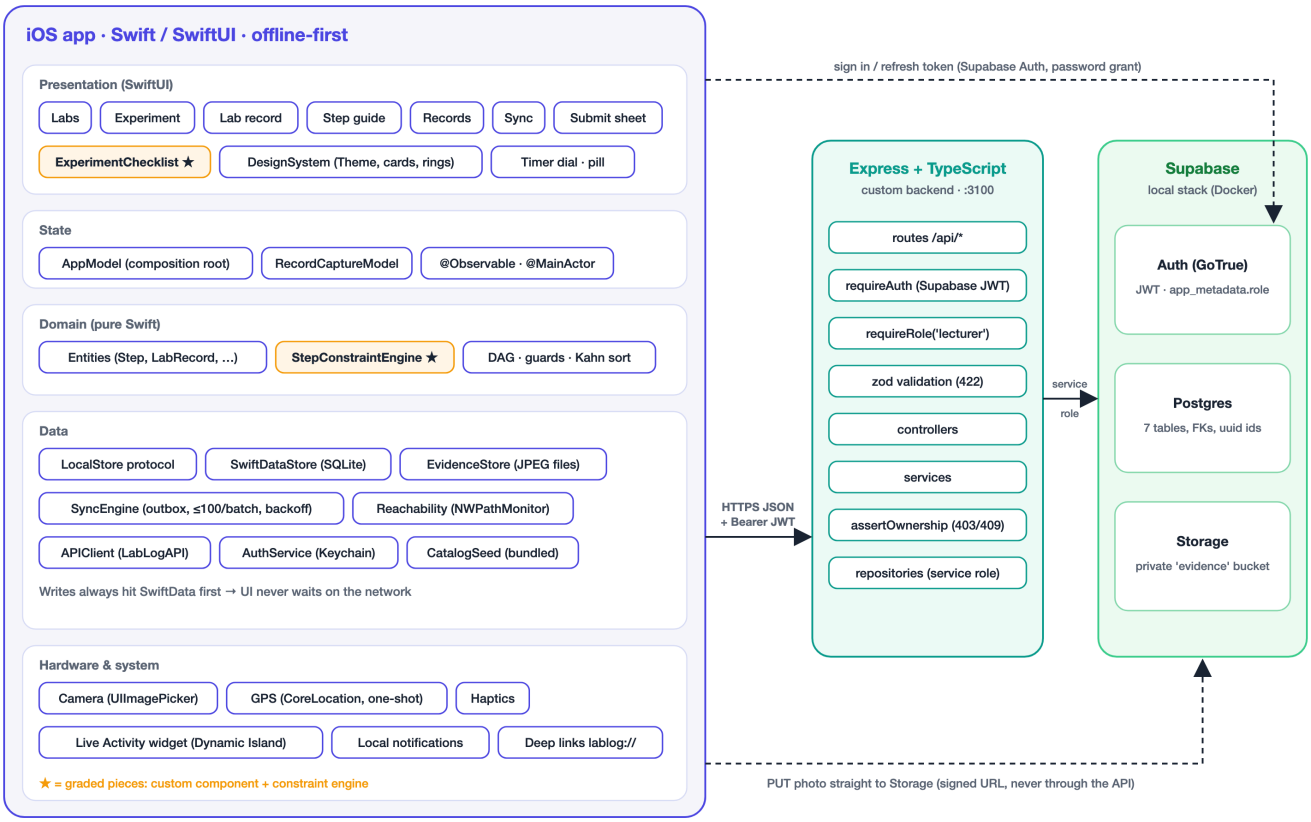


Figure 1 — Layers of the iOS app, the custom API, and Supabase. ★ marks the custom component and the constraint engine.

3.1 Layers and responsibilities

Layer	Contents	Responsibility
Presentation	SwiftUI screens, `ExperimentChecklist`, design system	Render state and turn taps into intents. No business rules.
State	`AppModel` (composition root), `RecordCaptureModel` (`@Observable`, `@MainActor`)	Hold screen state, call the domain and data layers, expose derived values (progress, next step, can-submit)
Domain	Entities, `StepConstraintEngine`	Pure Swift rules with no frameworks. The single source of truth for what a student may do.
Data	`LocalStore` → `SwiftDataStore`, `EvidenceStore`, `SyncEngine`, `Reachability`, `APIClient`, `AuthService`	Persistence, the outbox, networking and authentication
Hardware and system	Camera, CoreLocation, haptics, the Live Activity widget, notifications, deep links	Device capabilities, each behind a small wrapper
API (Express)	routes → middleware → controllers → services → repositories	Authentication, validation, ownership rules, data access
Supabase	Auth (JWT), Postgres, Storage	Identity, relational data, photo files

3.2 Major design decisions

All 23 decisions are recorded as ADRs (Context → Decision → Consequences) in `lablog-ios/docs/decision-records.md`. The ones that shape the design most:

ADR	Decision	Why
001	Native iOS with SwiftUI	First-class camera, location, Live Activities and SwiftData; a single demo platform
002	A custom Express API in front of Supabase, not direct database access from the app	A real service layer for validation, authorisation and business rules; the service-role key never ships in the app
003	The local store is the source of truth; the network is a background concern	The app must work fully offline in the lab
004	Client-generated UUIDs and idempotent upserts	Retrying after a dropped connection never creates duplicates
005	Photos uploaded through signed Storage URLs, not inside the JSON	Small sync payloads; photos never pass through the API
006 / 012	All step rules in one pure engine; step graphs validated with Kahn's algorithm	One place to reason about correctness; impossible experiments are rejected
008	Catalog bundled with the app, using the same UUIDs as the server seed	Usable offline on first launch, and records still link to the right server rows
009	A per-record `revision` counter guards in-flight edits	An edit made during a sync is never lost
013	Design system and a timeline checklist	Consistent visual language; the next step is always the most visible thing

ADR	Decision	Why
015	Timed steps as a Live Activity (Dynamic Island)	Students can leave the app during a wait
018	A local Supabase stack for end-to-end testing	Proves the full path on real Postgres, Auth and Storage
019	Lecturer role in `app_metadata`; ownership checks during sync	Closes record reading and record hijacking holes
020	In-app lecturer review, fetched live; the role only changes what the UI shows, and the API enforces it	Both roles demonstrable end to end; no trust placed in the client
021	The server address can change at runtime; the API points storage URLs at the host the app called	The demo works on any network, including a phone hotspot, with no rebuild
022	Discarding a synced draft sends a tombstone; the server deletes drafts only	Deletes are offline-first like every other write; submitted evidence is immutable
023	Photo retake (remove only until the step is done) and step notes; the payload carries the full photo set	Evidence can be corrected without breaking the photo guard or leaving stale files

3.3 Data flow between the layers

LabLog uses **unidirectional data flow**. Views send intents up, and models and stores push state down.

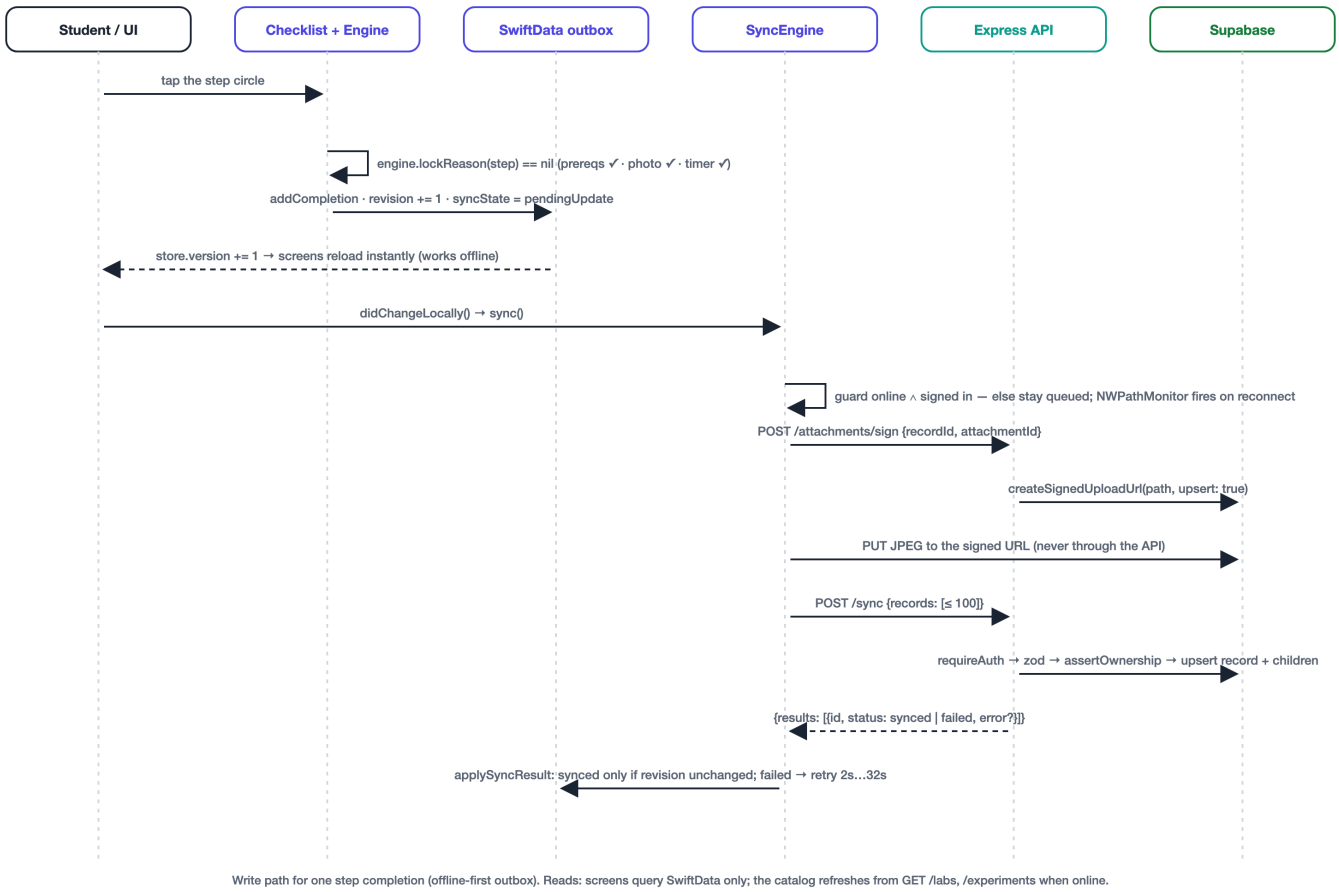


Figure 2 — What happens when a student completes a step: a local write first, then the outbox drains to the server.

Write path (Figure 2).

1. The student taps a step's circle in `ExperimentChecklist`.

2. The component asks `StepConstraintEngine.lockReason(for:context:)`. If the result is `nil`, it creates a `StepCompletion`.
3. The completion goes to `SwiftDataStore`. That increments the record's `revision`, marks it `pendingUpdate`, and bumps `store.version`.
4. Every screen that watches `store.version` reloads immediately, with or without a network.
5. `AppModel.didChangeLocally()` calls `SyncEngine.sync()`. If the phone is offline or signed out, the record simply stays queued; `NWPathMonitor` triggers the sync again on reconnect.
6. The engine uploads pending photos: `POST /attachments/sign`, then a `PUT` straight to Storage.
7. It then sends the records in batches of up to 100 (`POST /sync`). On the server, `requireAuth` checks the JWT, zod validates the payload, and `assertOwnership` blocks hijacking before the upserts run.
8. The per-item results come back, and `applySyncResult` marks a record `synced` only if its revision hasn't changed in the meantime.

Read path. Screens read only from SwiftData, through the `LocalStore` protocol, so the UI never waits on the network. When the phone is online and signed in, `SyncEngine.refreshCatalog()` pulls `GET /labs`, `/labs/:id/experiments` and `/experiments/:id`, validates each step graph, and updates the local catalog.

Patterns used (from the lectures):

- **MVVM with observable models**: views bind to `@Observable` models.
- **Repository pattern**: the `LocalStore` protocol on the app; repositories on the server.
- **Dependency injection**: the `AppModel` composition root builds every service once and passes it to views through the environment.
- **Protocol abstraction for testability**: `LabLogAPI` lets the sync engine be tested against a fake backend.
- **Layered API**: routes → controllers → services → repositories.

3.4 How the source is organised, and why

```
lablog-ios/
  LabLog/
    App/           entry point, AppModel (composition root), config, tabs
    DesignSystem/  Theme tokens + shared components (cards, rings, chips)
    Features/      one folder per screen: LabList, ExperimentList,
                   ExperimentDetail, RecordCapture (capture, step guide,
                   submit sheet), Records, SyncStatus, Auth,
                   Review (lecturer)
    Components/    ExperimentChecklist ★ (component, row, style), badges
    Domain/        Entities, Constraints/StepConstraintEngine ★ (pure Swift)
    Data/          Local (SwiftData store, catalog seed, evidence files)
                   Remote (API client, DTOs, Supabase auth, Keychain)
                   Sync (outbox, reachability)
    Hardware/      camera and location wrappers
    LiveActivity/  timer coordinator (activity + notifications)
  Shared/          compiled into app and widget (activity data, intent)
  LabLogWidgets/   widget extension: Dynamic Island + Lock Screen
  LabLogTests/     47 unit tests
  LabLogUITests/   5 end-to-end UI tests
  docs/            decision records, failure log, screenshots

lablog-backend/
  src/       routes → controllers → services → repositories,
             middleware (auth, roles, zod), dto
  db/        schema.sql, seed.sql, migrations/
  supabase/  local stack config
  test/      e2e.test.ts (14 tests)
  scripts/   local setup, demo users
```

Why this structure:

- **Feature folders** keep a screen's view and model together, so a change touches one place.
- **The domain layer has no framework imports**, so the most important rules can be unit-tested in milliseconds and reused by the app, the guide and the Live Activity intent.
- **The data layer sits behind protocols** (`LocalStore`, `LabLogAPI`), so SwiftData or the network can be swapped or faked.
- `Shared/` **exists only because the widget extension is a separate binary.** It holds the minimum both need.
- **The backend mirrors the same separation** (transport → logic → data), so each layer is testable and the business rules sit in services, not controllers.

3.5 Backend API

Method	Path	Access	Purpose
GET	`/api/health`	public	Liveness check
GET	`/api/labs` · `/api/labs/:id/experiments` · `/api/experiments/:id`	student	Catalog, including the step graph and guides
POST	`/api/attachments/sign`	student	Signed upload URL for a photo (deterministic path, upsert)
POST	`/api/sync`	student	Batch upsert of records and their children; result per record. `deleted: true` deletes a draft (tombstone); photos missing from a record's set are removed
GET	`/api/records?experimentId=`	**lecturer**	Submitted records with student email and 1-hour signed photo URLs

Data model (Postgres):

- Catalog: `labs` 1–* `experiments` 1–* `steps`. Each step has `prerequisite_step_ids uuid[]`, `requires_photo`, `min_duration_seconds` and its guide content.
- Records: `lab_records` (owned by `auth.users`) 1–* `step_completions`, `observations` and `attachments`.
- All record ids are client-generated UUIDs.
- Storage URLs that the API hands to the app (signed uploads and lecturer photo links) use whatever host the app called. The Mac's address can then change without touching the backend (ADR-021).

---

4. How offline access works

LabLog is **local-first**. The phone is the source of truth while the student works, and the server catches up.

Technique	How it is implemented
Usable offline from first launch	`CatalogSeed` bundles 2 labs, 3 experiments and 13 steps with the same UUIDs as `seed.sql`, versioned so updates reach existing installs
Local persistence	SwiftData models mirror the domain structs; every action writes locally first
Outbox	Each record carries a local `syncState` (`pendingCreate`, `pendingUpdate`, `synced`, `failed`); every non-synced record is in the outbox
No lost edits	A `revision` counter: a sync result marks a record synced only if no edit happened while the request was in flight
Safe retries	Client UUIDs and upserts by id; the photo path is deterministic and uploads overwrite (`upsert: true`)
Photos offline	`EvidenceStore` keeps downscaled JPEGs (1600 px, quality 0.7) in Application Support, by relative file name, until uploaded
Reconnect detection	`NWPathMonitor`: the offline → online transition triggers a catalog refresh, then a sync

Technique	How it is implemented
Retry policy	A failed sync retries with exponential backoff (2, 4, 8, 16, 32 s); reconnecting or *Sync now* retries immediately
Efficient sync	Photos upload first; records go in batches of up to 100; the server reports success or failure per record, so one bad record doesn't block the rest
Login never blocks lab work	Sign-in is only needed to sync; the session lives in the Keychain and refreshes a minute before expiry
Deletes offline too	A discarded draft the server already has becomes a **tombstone**: hidden everywhere, kept in the outbox, sent as `deleted: true`, purged once the server confirms. A draft never sent is just deleted (ADR-022)
Edits as full state	A record's payload lists its complete photo set, so a retaken or removed photo is dropped on the server too (rows and files)
Any network	The server address can be changed in the app at runtime; the session survives because it's the same backend (ADR-021)

What the student sees: an *Online/Offline* pill with the number of pending records on every screen, a sync state on every record, and a Sync screen that leads with one clear status ("Working offline", "3 records ready to sync", "Everything is synced"). Its main button is whatever unblocks sync next (*Sign in* or *Sync now*).

Evidence:

- **Unit tests:** 15 store tests and 8 sync-engine tests cover outbox rules, the revision guard, photo-before-record ordering, per-item failures, tombstones, photo removal and notes, and the offline and signed-out behaviour.
- **Backend tests:** 14 end-to-end tests against real Postgres, Auth and Storage, including a tombstone retried twice, a retake replacing the stored photo, and storage URLs following the caller's host.
- **UI test:** it signs in, completes steps with a photo, and waits for "Everything is synced". The record, both step completions and the photo were then confirmed in the database (Figure 9).

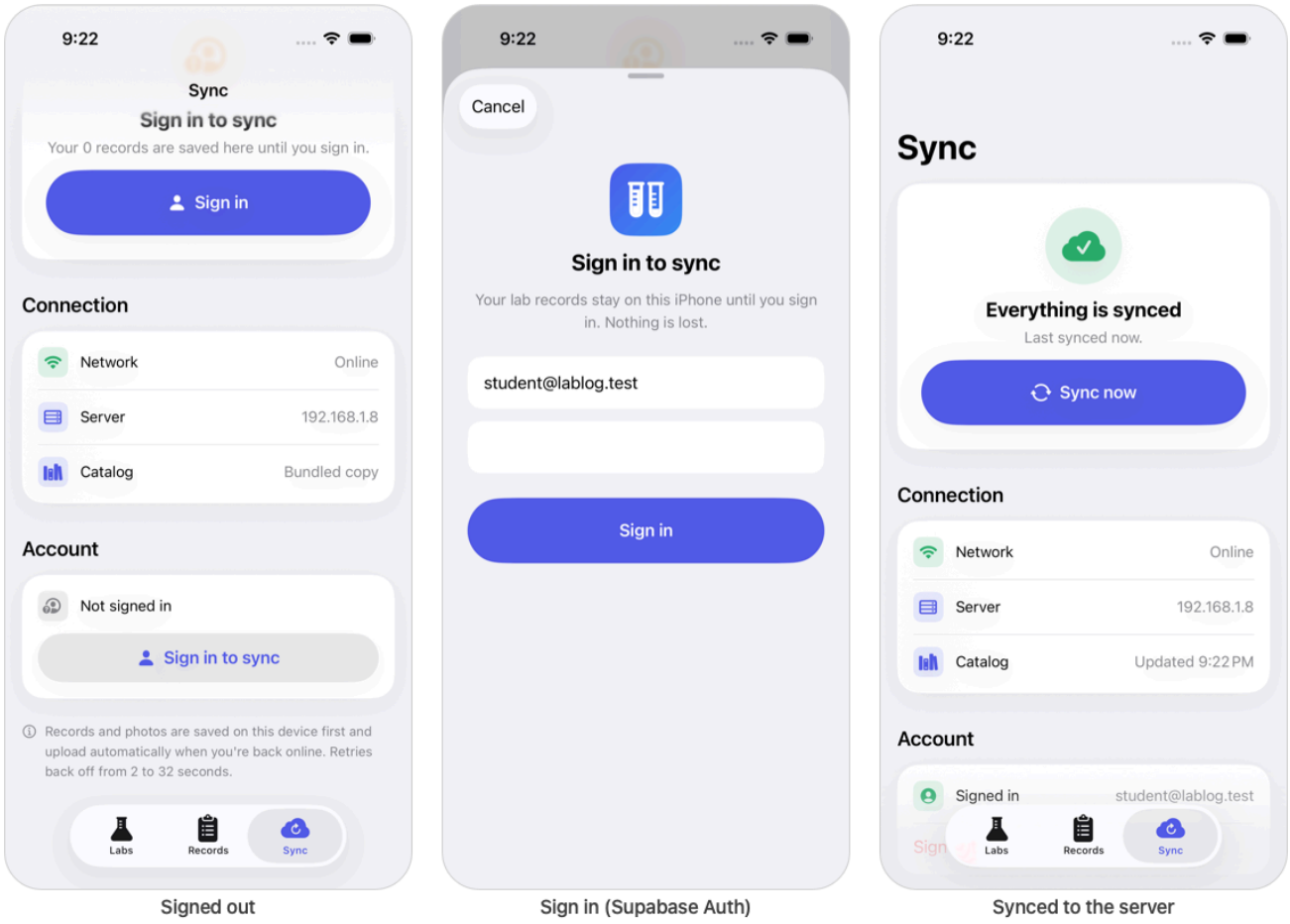


Figure 9 — Signed out; signing in through Supabase Auth; the outbox drained to the server.

5. Constraint design algorithms and principles

5.1 Model

An experiment is a **directed acyclic graph of steps**. Each step lists its prerequisites (`prerequisiteStepIds`) and may carry **guard conditions**:

- `requiresPhoto`: a photo must be attached to this step.
- `minDurationSeconds`: a minimum wait, measured from the latest prerequisite's completion.

The seeded *Acid–Base Titration* experiment is a real branching graph. Steps 1 and 2 can be done in either order, and Step 4 needs **both** Step 1 and Step 3.

5.2 The unlock algorithm

```
lockReason(step, ctx):
  missing ← step.prerequisites − ctx.completed
  if missing ≠ ∅                          → prerequisitesIncomplete(missing)   // precedence
  if step.minDuration ≠ nil:
     anchor ← max(completionTime[p] for p in step.prerequisites)           // latest prereq
     remaining ← ceil(minDuration − (now − anchor))
     if remaining > 0                      → timerPending(remaining)          // time guard
  if step.requiresPhoto ∧ step ∉ ctx.photos → needsPhoto                     // evidence guard
  return nil                                                                 // unlocked
```

The order of the checks is deliberate. Precedence is reported first, so the student always sees the real blocker. The timer is anchored on the *latest* prerequisite, so a parallel branch can't shorten a wait. The engine is a pure `struct`: given the same steps, completions, photos and time, it returns the same answer, which makes it easy to test.

The engine also provides:

- `reachableSteps(…)`: what the student can work on next. It drives "Next: Step 3 · …".
- `isComplete(…)`: whether every step is done. It gates submission.
- `message(for:)`: a human-readable reason, such as "Complete Step 1, Step 3 first" or "Wait 2:30".

5.3 Graph validation (Kahn's algorithm)

Step graphs come from the server, and a typo could create a cycle (A needs B, B needs A) or a prerequisite that doesn't exist. Either would make an experiment impossible to finish.

`validate(_:)` reports self-dependencies, unknown prerequisites and cycles. Cycles are found with **Kahn's topological sort**: repeatedly take a step whose prerequisites are all placed, lowest `order` first so the result is deterministic. Any steps never placed lie on or behind a cycle. `refreshCatalog()` skips an experiment that fails validation, so a student never receives an impossible one.

As written, it costs O(V·(V+E)), because it rescans the step list each round. That is trivial for a lab experiment (≤ 20 steps). A priority queue would make it O((V+E) log V).

5.4 Principles applied

1. **Precedence constraints.** A topological order is enforced, so Step 4 can't be reached before Steps 1 and 3.
2. **Guard conditions.** Evidence and time preconditions are first-class, not ad-hoc `if`s.
3. **Invariant centralisation (single source of truth).** The checklist, the step guide, the Dynamic Island's *Mark done* intent and the submit gate all ask the same engine, so no screen can disagree with another.
4. **Fail-safe, explanatory UI.** Locked steps stay tappable and say *why* they're locked (with a warning haptic); the guide lists the blockers and jumps to them.
5. **Defence in depth.** The server re-validates every payload with zod (422 on bad input), checks ownership (403 or 409), and limits batch size.
6. **Validate before use.** Graph validation rejects an impossible configuration before it reaches a student.
7. **Invariants survive edits.** "A completed photo step has a photo" holds after the fact as well. A completed step's photo can be retaken but not removed, and submitted records refuse every edit, in the

store and (for deletes) on the server.

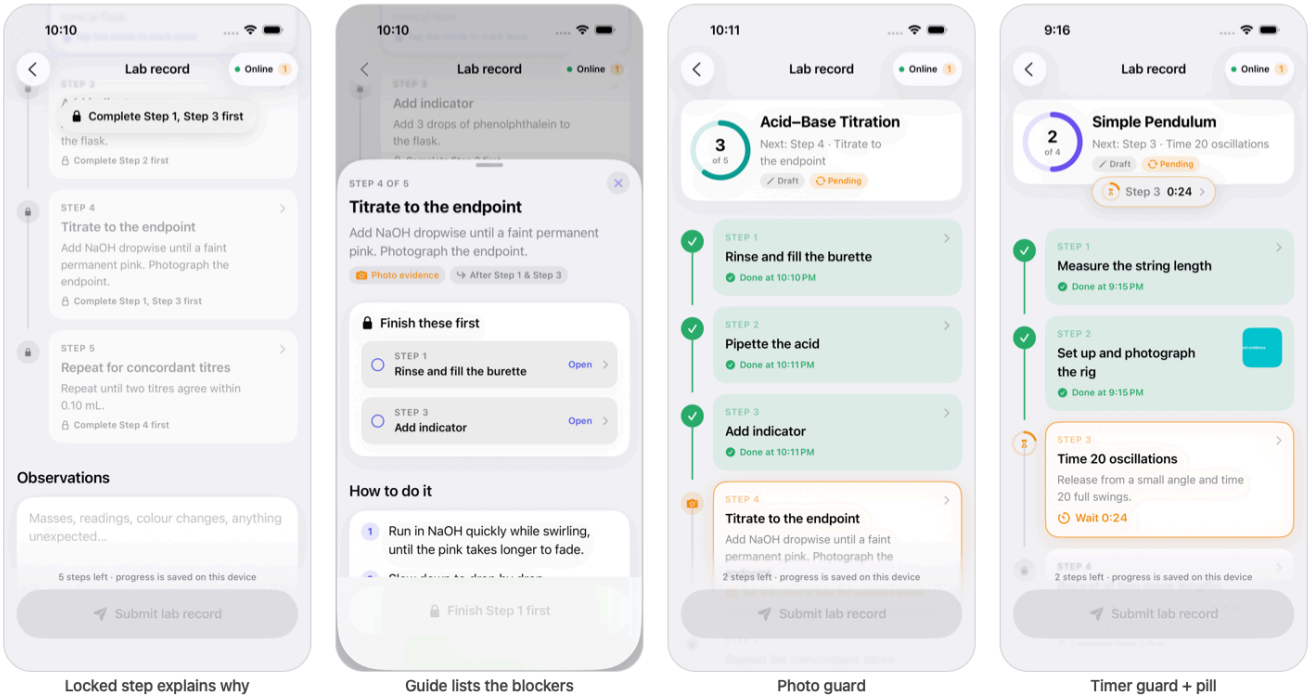


Figure 4 — A locked step explains itself; the guide lists blocking steps; photo guard; timer guard with a live countdown.

5.5 Designing for a constrained device

The app also follows the principles of designing for limited battery, network, storage and attention:

Constraint	Design response
Unreliable network	Local-first writes, outbox, batching (≤ 100), per-item results, backoff instead of tight retries
Bandwidth	Photos downscaled to 1600 px and JPEG 0.7 (about 100 KB); uploaded directly to Storage, never through the API
Battery	No polling: sync is driven by events (`NWPathMonitor`, local changes); the timer view only ticks while a timer runs; the Live Activity countdown is drawn by the system with no app updates; GPS is a one-shot fix at 100 m accuracy with a 10 s timeout
Storage and memory	Evidence kept by relative file name; thumbnails generated with `preparingThumbnail`, not full-size images
Privacy and permissions	Camera, location and notification permission are asked only when the feature is first used (submit, photo, first timer), never at launch
User attention	One primary action per screen; capture mode hides the tab bar; Reduce Motion is respected

Tests: 12 constraint-engine unit tests. They cover multi-parent prerequisites, guard order, timer anchoring, messages, reachability, topological order, cycle detection, dangling and self edges, and the validity of every bundled experiment.

---

6. Native hardware integration

Capability	Implementation	Purpose	If unavailable
Camera	`UIImagePickerController` wrapped in `CameraPicker`; images saved by `EvidenceStore`	Photo evidence attached to a specific step; a photo automatically completes a photo-guarded step when the step is otherwise unlocked	Falls back to the photo library (Simulator); permission text set in `NSCameraUsageDescription`
GPS	`LocationService` (CoreLocation): one-shot `requestLocation`, 100 m accuracy, 10 s timeout, async continuation	Location stamp on submission, shown on a MapKit receipt	The record still submits with no location ("Not available")
Haptics	SwiftUI `sensoryFeedback`	Success when a step completes, warning when a locked step is tapped, a level change when a timer ends, success on submit	No effect
Live Activities / Dynamic Island	Widget extension with `ActivityKit`; `Text(timerInterval:)` and `ProgressView(timerInterval:)`	The countdown continues outside the app; *Mark done* runs a `LiveActivityIntent` in the app process, still checked by the engine	Skipped if Live Activities are disabled
Notifications	`UNUserNotificationCenter`, time-interval trigger	"Step 3 is ready" when the wait ends; tapping it deep-links to the step guide	Skipped if permission is denied



Figure 7 — The compact Dynamic Island (ring and countdown); expanded (step, progress, ready time, Guide and Mark done); the Lock Screen Live Activity with the "ready" notification.

7. UI/UX flow

7.1 Navigation

There are three tabs (**Labs · Records · Sync**), plus a fourth, **Review**, for lecturers. The stacks are shallow: Labs → lab → experiment briefing → lab record. A student is never more than three taps from the checklist. While capturing, the tab bar hides (focus mode), and *Submit* is pinned to a floating bottom bar.

7.2 Screen by screen

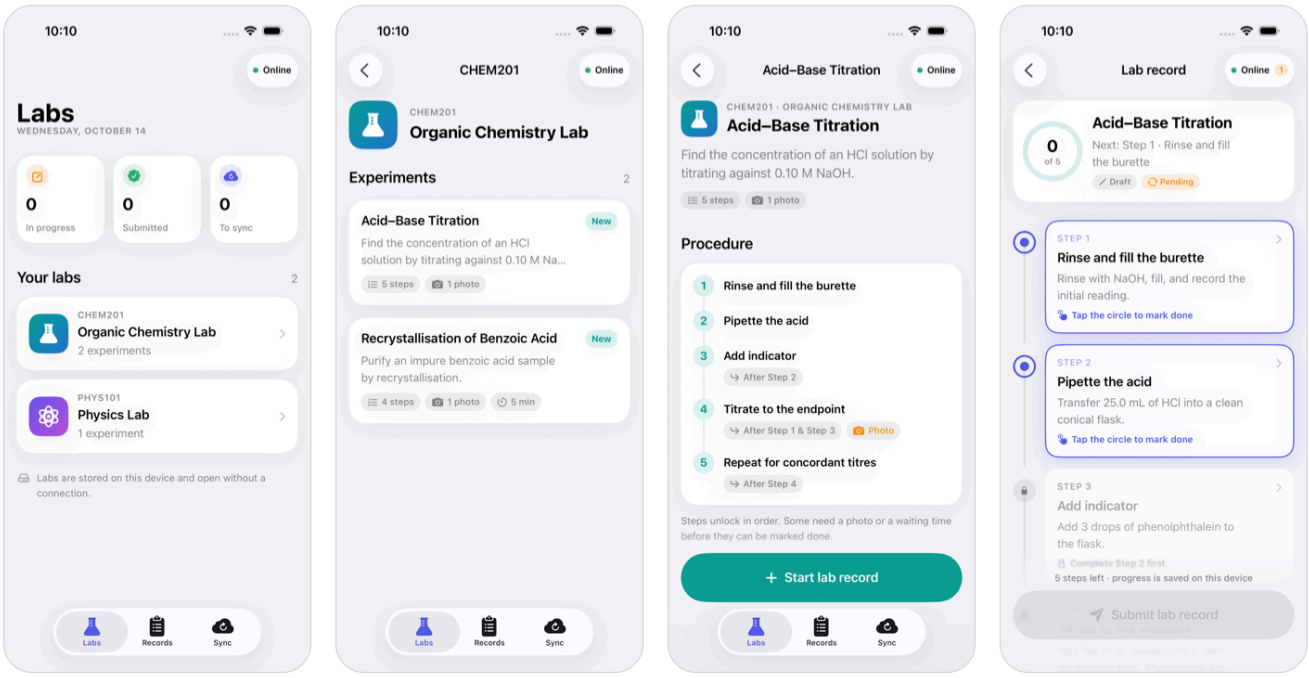


Figure 3 — The Labs dashboard (date, stats, lab cards); a lab's experiments with their demands (steps, photos, waits) and progress; the experiment briefing with the procedure and its constraints, plus a sticky "Start lab record" button; a new record.

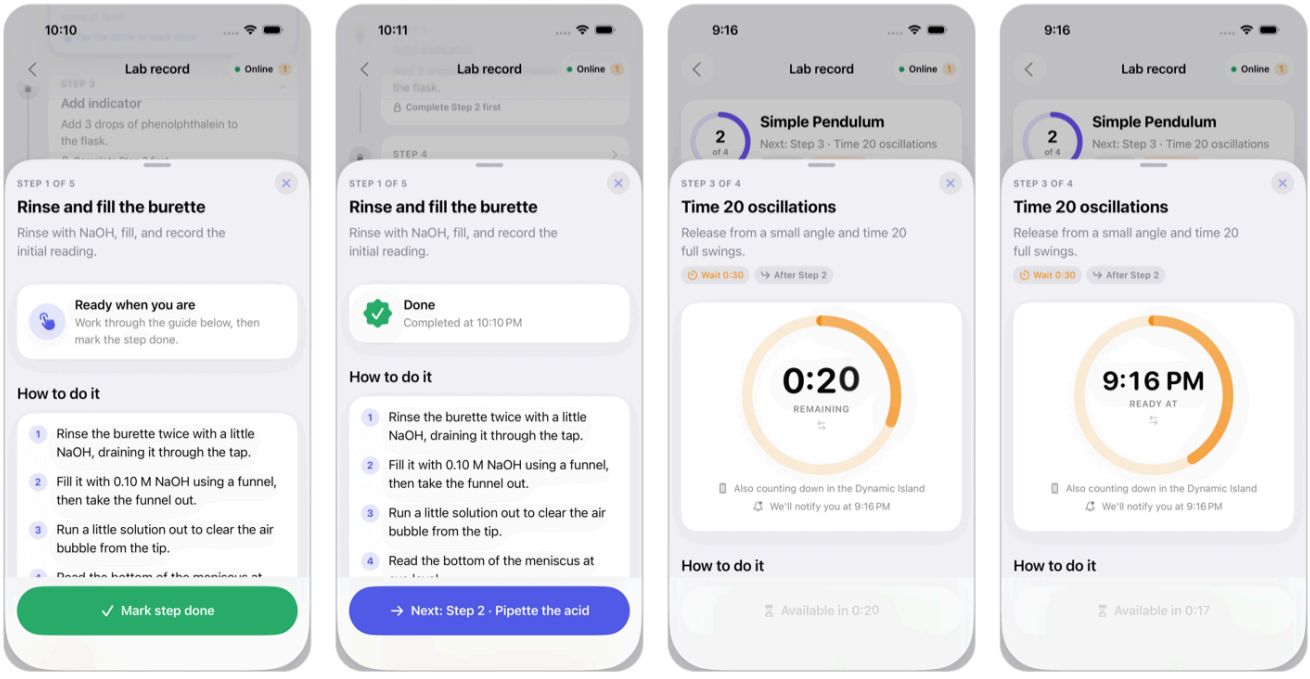


Figure 5 — The step guide (how-to, safety, expected result, one primary action); after completion it offers the next step; the timer dial (tap to switch between remaining time and ready-at time).

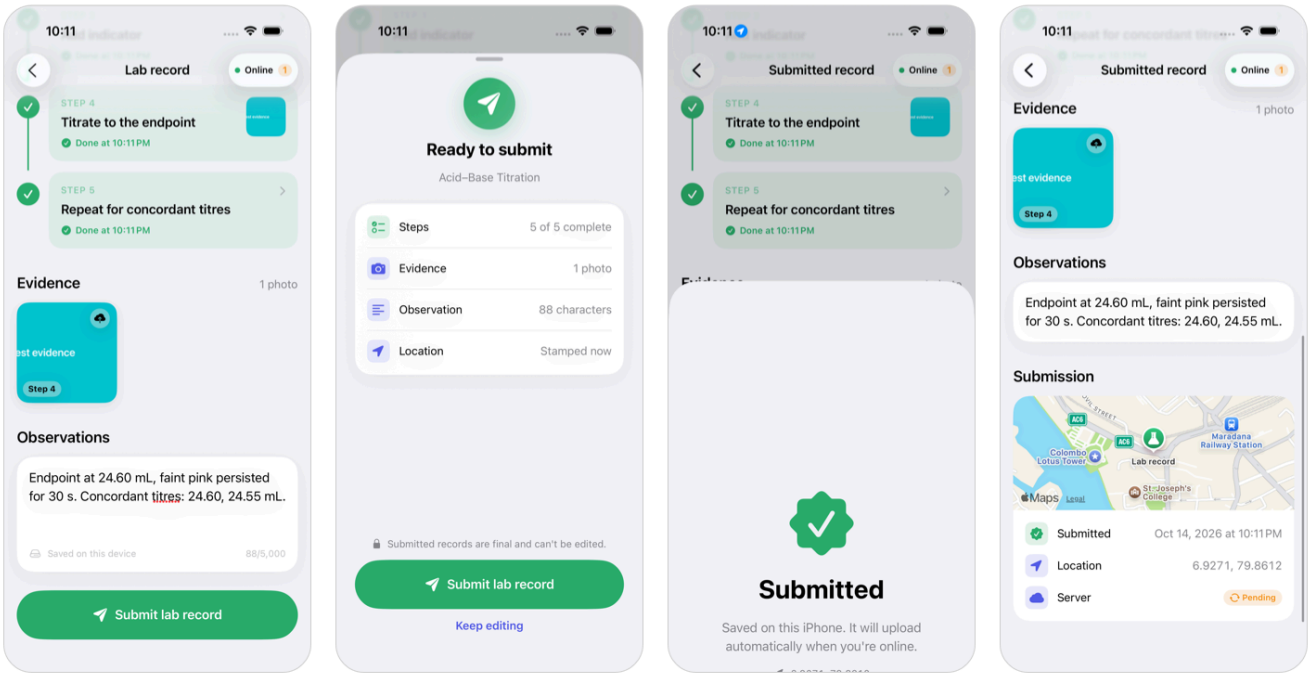


Figure 6 — All steps done; the review-and-submit sheet summarising what will be handed in; the animated "Submitted" state; the read-only receipt with a GPS map.

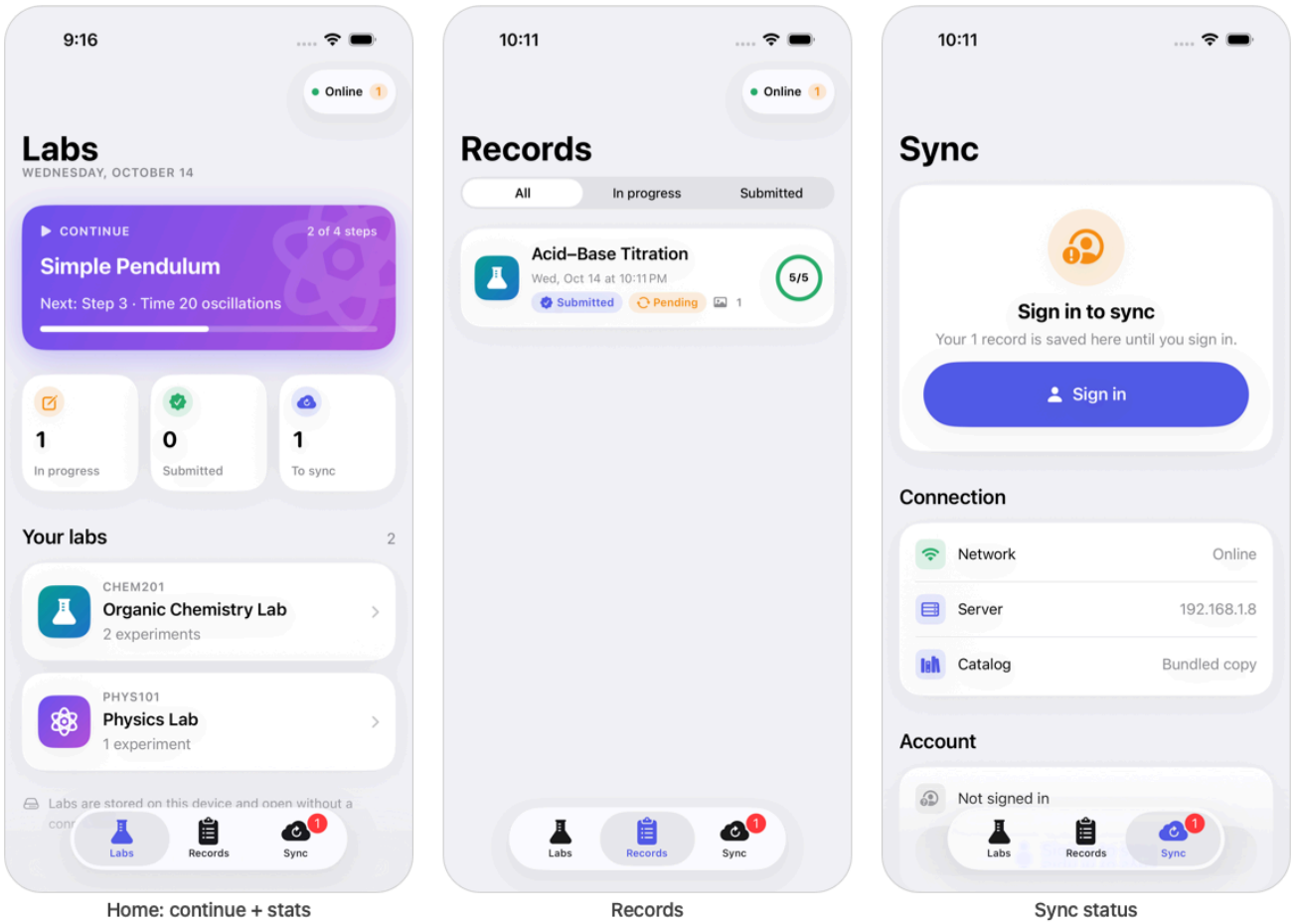


Figure 8 — Home with a "Continue" card for the current draft; Records with filters, progress rings and sync badges; the Sync screen.

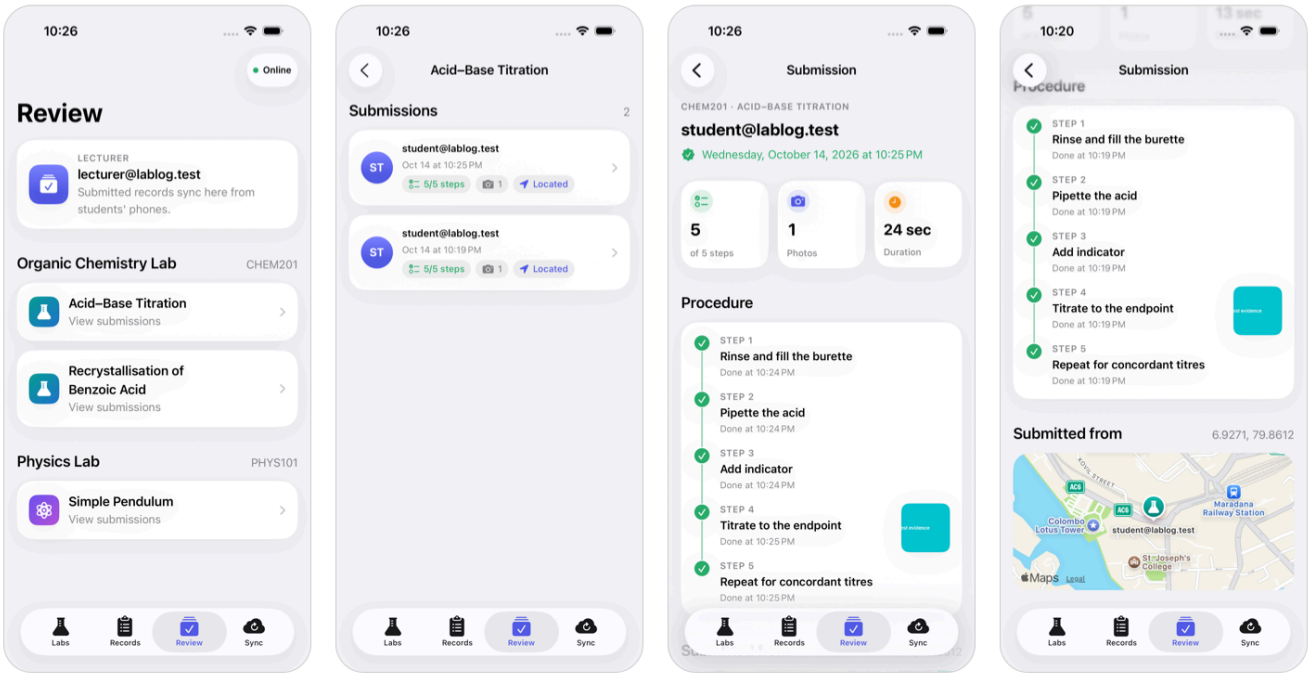


Figure 10 — Signed in as a lecturer: the Review tab; an experiment's submissions; a submission's detail (steps done, photos, time taken, the step timeline with completion times and evidence); and where it was submitted.

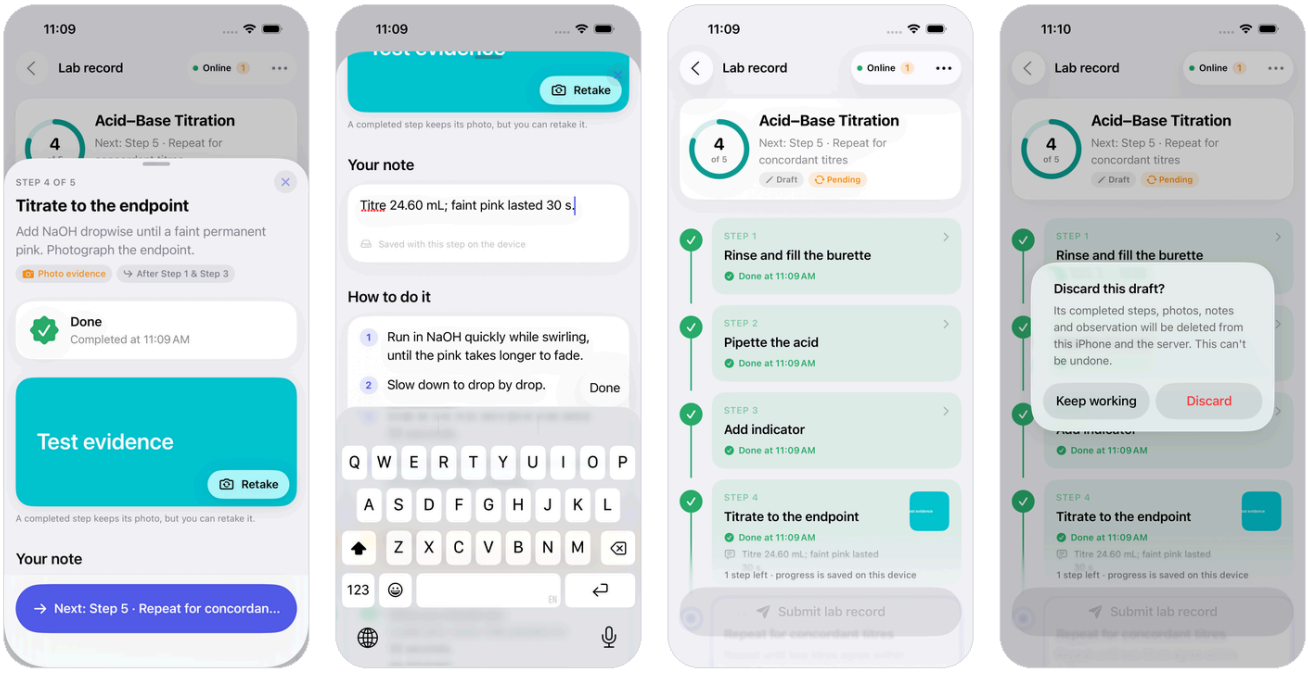
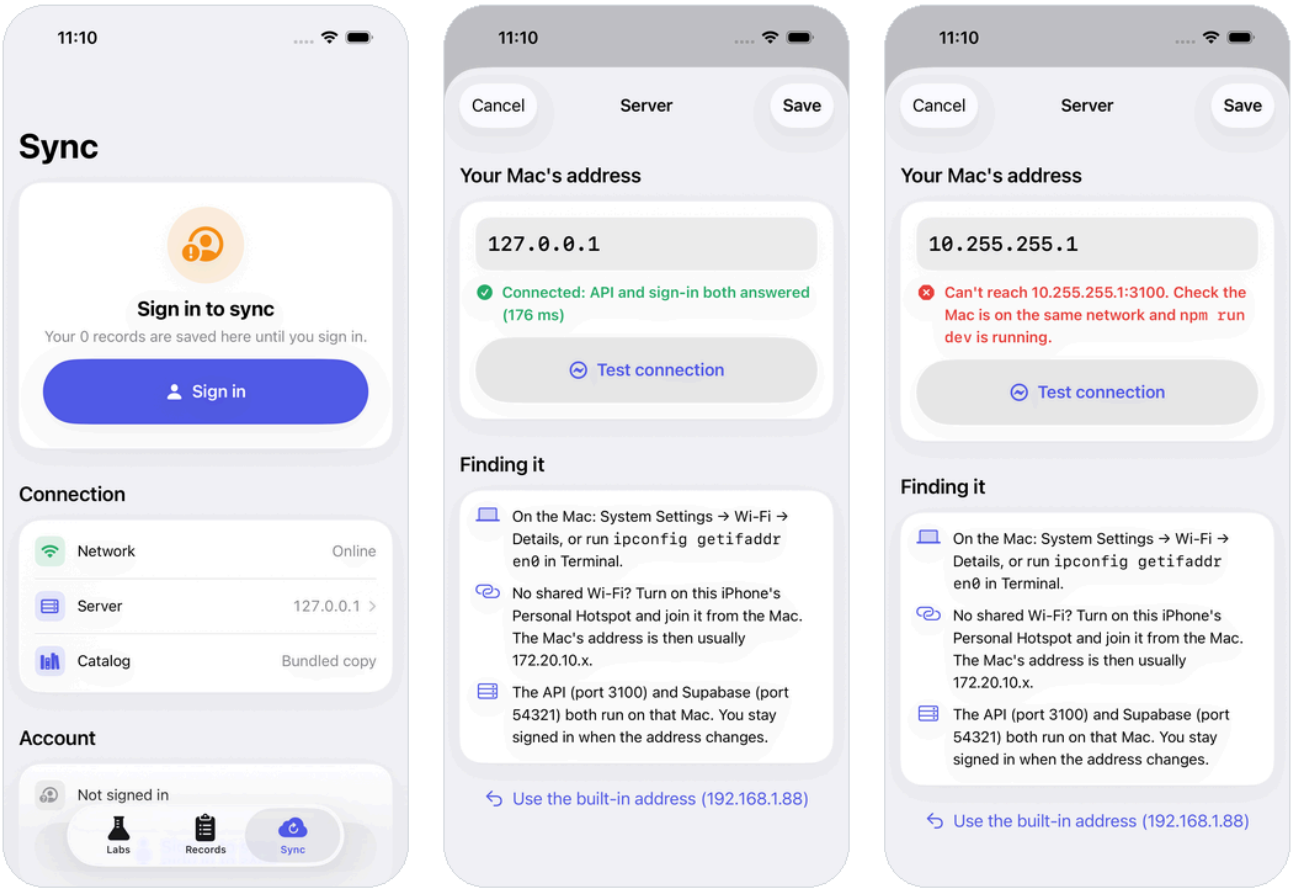


Figure 11 — Retake sits on the photo itself (a completed step keeps a photo, so there is no Remove); a note on the step, with the action bar out of the keyboard's way; the note under its step in the checklist; discarding the draft, with a confirmation that says what will be deleted.

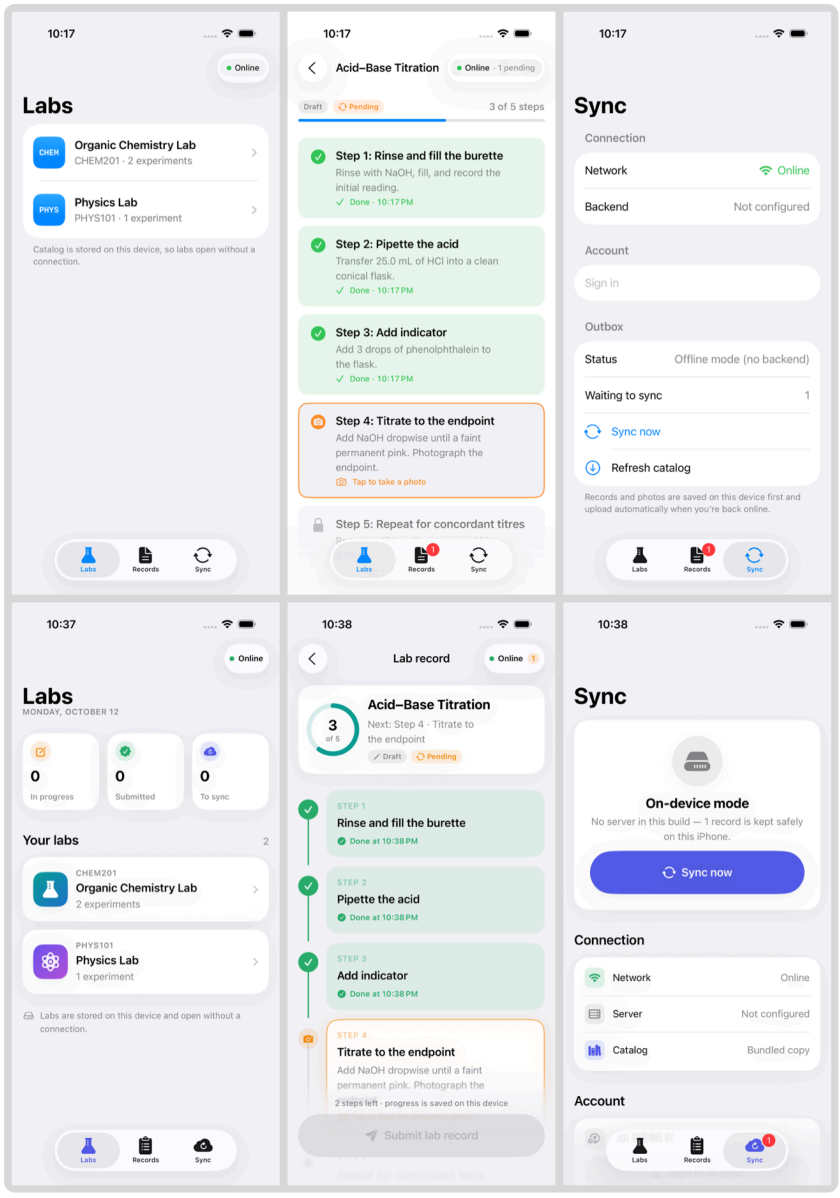


Sync: the server row | Address answers | Address doesn't answer

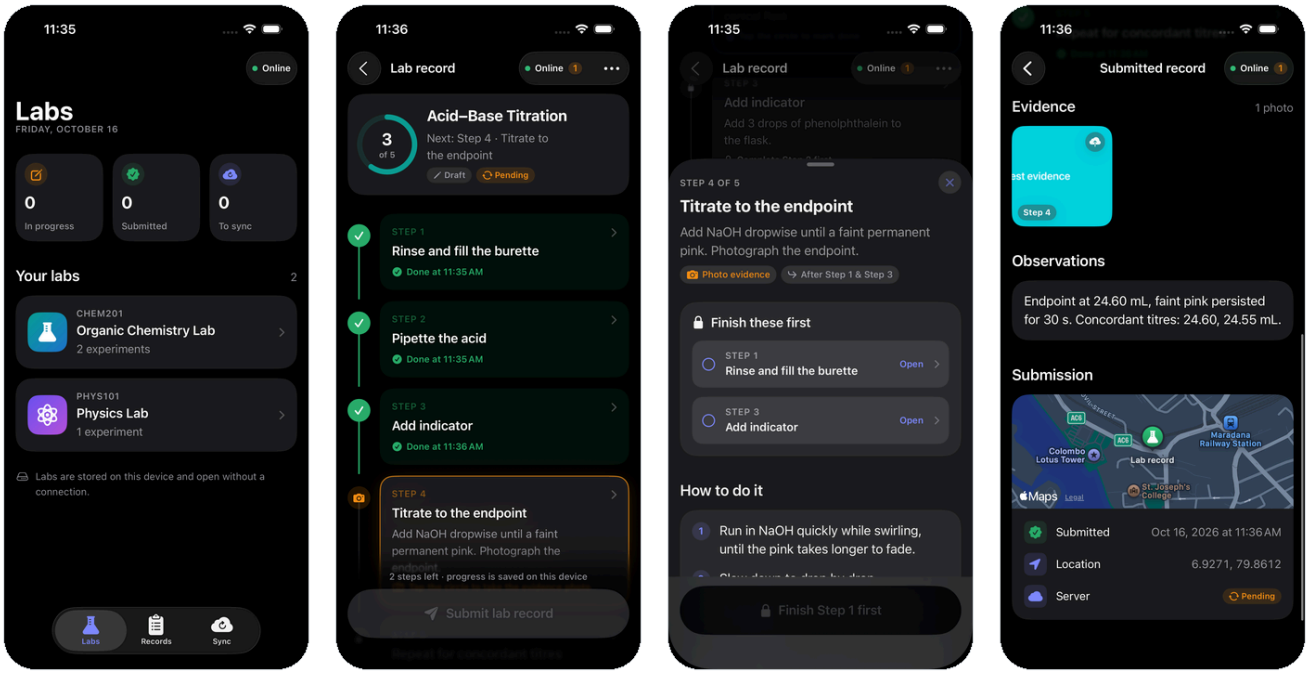
Figure 12 — Sync shows which server the app is using; the Server sheet tests an address against both the API and sign-in before saving; an address that doesn't answer says what to check.

7.3 Why these UI/UX choices

- **The next action is always the most visible thing.** Available steps are outlined and lifted, locked steps recede, and finished steps collapse to one line.
- **Constraints are explained, not just enforced.** Every lock has a reason, and the guide links to the blocking step.
- **Built for a student mid-experiment** (gloves, divided attention). The tap targets are large, typing is minimal (the photo completes the step, and the email field focuses itself), haptics confirm actions, and the timer follows the student out of the app.
- **Irreversible actions are confirmed with context.** The review sheet shows exactly what is being submitted, rather than a generic "Are you sure?".
- **Offline is never alarming.** The Online/Offline state and the pending count are always visible, with a reassuring message ("saved on this device").
- **Follows Apple's Human Interface Guidelines.** System fonts and SF Symbols; critically damped springs, with bounce kept for completions; materials; dark mode; VoiceOver labels and hints; Reduce Motion support.
- **Original assets.** The app icon and every illustration are drawn in code, and the colours come from my own palette.



The first working build (top) used stock lists. The redesign (bottom) introduced the design system and the timeline checklist (ADR-013).



Labs | Checklist | Step guide | Submitted record

Figure 13 — The same screens in dark mode. The app follows the system appearance: colours are semantic (system backgrounds and label colours), white text only sits on coloured fills, and card shadows are dropped in the dark. I checked this by running the UI tests with the simulator in dark appearance (16 Oct). All three passed, and all 29 screenshots were reviewed for contrast, with nothing to fix.

---

8. Custom components

8.1 `ExperimentChecklist` (the main custom component)

A reusable SwiftUI view that renders an experiment as a **vertical timeline** and enforces the step rules through an injected `StepConstraintEngine`. The component never decides locking itself; it asks the engine.

Properties (inputs)

Property	Type	Meaning
`steps`	`[Step]`	The steps to render (shown in `order`)
`completions`	`Binding<[StepCompletion]>`	Two-way bound completion state
`recordId`	`UUID`	The record that new completions belong to
`photoStepIds`	`Set<UUID>`	Steps that already have a photo
`photoThumbnails`	`[UUID: UIImage]`	Optional evidence thumbnails shown on their step
`isEditable`	`Bool`	`false` for a submitted record (read-only)
`engine`	`StepConstraintEngine`	The rules (injectable, so it can be swapped in tests)
`style`	`ChecklistStyle`	Theme: spacing, radius, node size, tints, collapse behaviour

Events (outputs)

Event	Fires when
`onStepCompleted(Step, StepCompletion)`	A step was completed; the parent persists it, and successors re-evaluate automatically
`onPhotoRequested(Step)`	The circle of a photo-guarded step was tapped; the parent opens the camera
`onLockedStepTapped(Step, String)`	A locked step was tapped; the reason text is passed for display
`onStepSelected(Step)?`	A step card was tapped; the parent opens the step guide. If `nil`, a card tap behaves like a circle tap

A completion's `note` (in the `completions` binding) is shown under its step. The row view, `ChecklistRow`, gained a `note` property for it, so the component needed no new input.

Row states: *locked* (with reason), *available*, *needs photo*, *timer pending* (live ring and countdown), *completed* (with time and thumbnail).

How it works:

1. Each render builds a constraint context from `completions`, `photoStepIds` and the current time, and maps every step to a row state through the engine.
2. A `TimelineView` re-renders once a second, but only while a timer guard is counting down.
3. When a countdown ends, the component fires a haptic.
4. Tapping a node runs the quick action (complete, request a photo, or explain the lock); tapping a card selects the step.
5. Completing a step appends to the binding and raises `onStepCompleted`, and the next render unlocks the successors.

Rows have VoiceOver labels, values and hints. The node and the card are separate accessibility elements.

```
ExperimentChecklist(
    steps: model.steps,
    completions: $model.completions,
    recordId: model.recordId,
    photoStepIds: model.photoStepIds,
    photoThumbnails: thumbnails,
    isEditable: model.isEditable,
    onStepCompleted: { _, completion in model.persist(completion) },
    onPhotoRequested: { step in requestPhoto(for: step) },
    onLockedStepTapped: { _, reason in show(reason) },
    onStepSelected: { step in guide = GuideTarget(id: step.id) }
)
```

8.2 Other custom components

Component	Purpose
`ChecklistRow`	One timeline row: status node, connector, card; five visual states

Component	Purpose
`StepGuideSheet`	The guide: blockers you can jump to, interactive `TimerDial`, the single next action
`TimerDial` and `LiveTimerPill`	A countdown ring (tap to switch to ready-at time) and a floating live pill under the header
`SubmitReviewSheet`	Review → submitting → success, with an animated confirmation
`ProgressRing`, `DisciplineIcon`, `StatTile`, `MetaChip`, `InfoRow`, `SyncStateBadge`, `ConnectivityPill`	Design-system building blocks
`StepTimerLiveActivity`	Dynamic Island (compact, minimal, expanded) and Lock Screen layouts
`ReviewView`, `SubmissionsView`, `SubmissionDetailView`	The lecturer's review flow: stats, step timeline, photos from signed URLs, location map

9. Decision records, failure logs and AI use

9.1 How the process was recorded

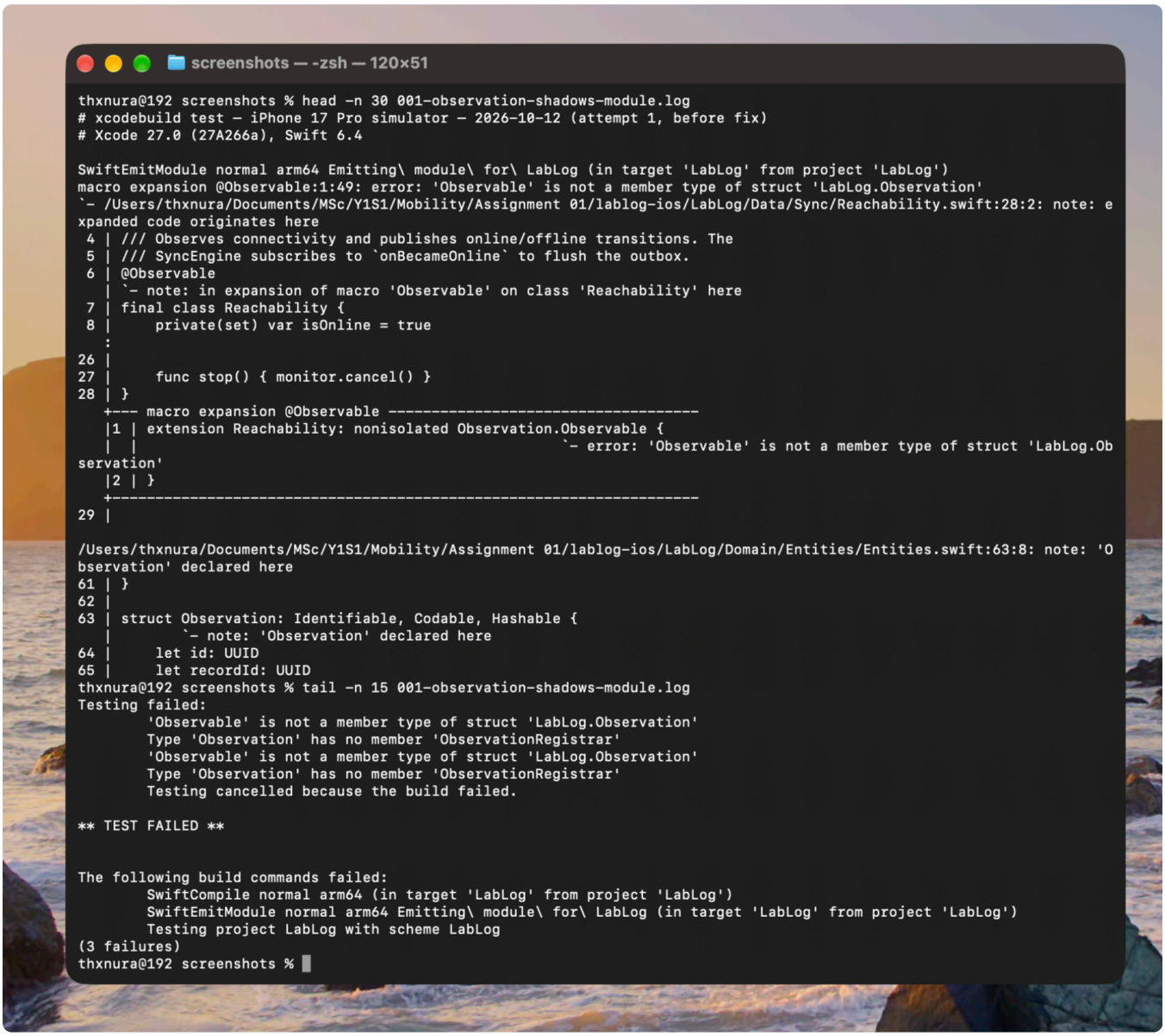
- **Decisions:** ADR-001 to ADR-020 in `lablog-ios/docs/decision-records.md` (summary in §3.2).
- **Failures:** `lablog-ios/docs/failure-log.md`, with the exact error text, an image, the cause and the fix. Raw logs sit next to each image in `docs/screenshots/`.
- **AI interactions:** every prompt is quoted exactly as typed (§9.3), with what it produced and whether it worked.

9.2 Failure log

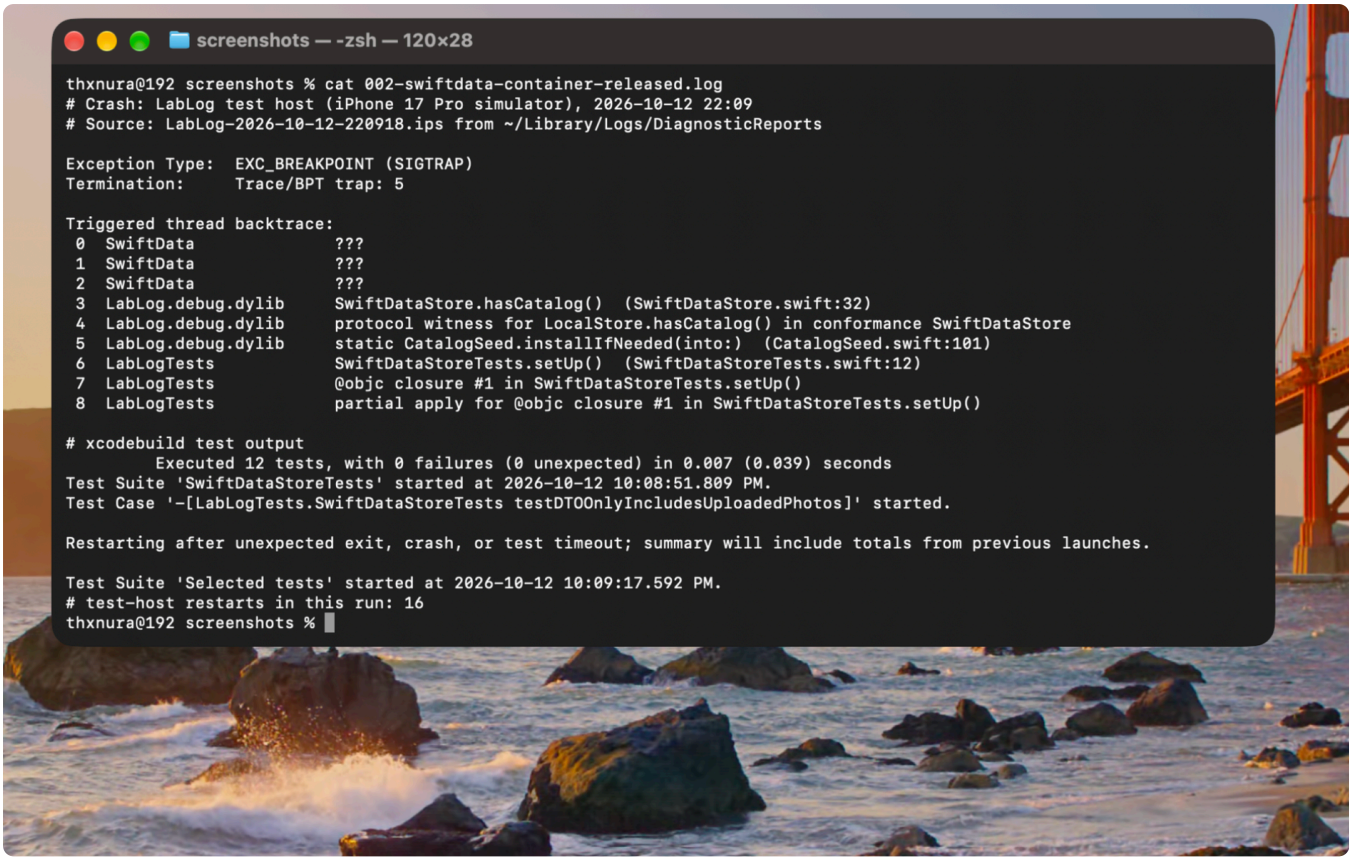
#	Date	Failure (exact error)	Cause	Fix
001	12 Oct	`'Observable' is not a member type of struct 'LabLog.Observation'`	My domain type `Observation` shadowed Apple's Observation module, which the `@Observable` macro expands into	Renamed it `LabObservation`
002	12 Oct	`EXC_BREAKPOINT (SIGTRAP)` in SwiftData; the test host crashed 16 times	The store kept the `ModelContext` but not its `ModelContainer`, which was freed	The store holds the container strongly
003	12 Oct	Badge wrapped as "Pend-ing", then showed only an icon; a row 4× too tall	SwiftUI `Label` adapts its layout to its container	Explicit `HStack { Image; Text }`
004	12 Oct	`XCTAssertTrue failed` — the UI test tapped the floating Submit bar	The test tapped an element's centre, which sat under the bar	Scroll into the clear band, then tap

#	Date	Failure (exact error)	Cause	Fix
005	12 Oct	Both UI tests: `XCTAssertTrue failed`; the Close button carried the id `step-guide`	A container's accessibility identifier overrode its children's	Identifiers on leaf elements only; combine before overlaying
006	12 Oct	`Failed to tap "start-record"` and a missing timer pill	A tap during a navigation push; the island press deep-linked into the guide	Wait for screens; handle the guide
007	12 Oct	The submit dialog appeared as a popover pointing at a step card	A `confirmationDialog` anchored to a large container	A custom Review & submit sheet
008	14 Oct	`listen EADDRINUSE: address already in use :::3000`	Another project's dev server held port 3000	The local API runs on 3100
009	14 Oct	Flaky sign-in test (three different errors)	Tab-bar overlap, sheet animation timing, keyboard focus	Auto-focus the email field; waits; hierarchy dump on failure
010	14 Oct	Receipt showed "Location: Not available"; then `XCTAssertTrue failed` twice	The restarted simulator had no simulated location (the app correctly submitted without one); a dropped tap during a transition; the test missed the 1.6 s success state	Tests pin the location (`XCUIDevice.location`), retry navigation taps, and watch for success and prompts together. GPS confirmed: "6.9271, 79.8612"
011	14 Oct	Lecturer UI test: tab tap ignored after launch; `XCTAssertTrue failed` (Submit disabled)	Cause of the ignored tap not found; Step 5 tapped while completed rows were re-laying out	Verified tab switching (title check, one retry); confirm each step completed
012	16 Oct	`XCTAssertTrue failed` waiting for `step-note`; the UI hierarchy showed Step 5's guide	The new Retake button sat under the sheet's floating "Next: Step 5" button, so the tap went to Next (the same class as 004, and a real UX flaw)	Retake and Remove float on the photo itself; the test asserts Retake is hittable and the guide stays on Step 4
013	16 Oct	(Tooling) My keystrokes "ow" + Return landed in TermShot's Terminal window: `zsh: command not found: ow`	TermShot took the keyboard focus while I was typing, and its window ended in a live shell	It waits for keyboard and mouse to go idle, ends in a real shell with an empty keymap, and retakes the shot if a key was pressed. The faulty capture was overwritten, so there is no image
014	16 Oct	`Failed to tap "Discard" Button: … Multiple matching elements found`	iOS 26 exposes an alert's button to XCUITest twice	The test takes the first match inside the alert
015	16 Oct	Both sign-in UI tests: `XCTAssertTrue failed - sign-in against Supabase Auth`	The router moved the Mac from 192.168.1.8 to .88 mid-session; the app was built with the old address (the demo-day risk, for real)	UI tests launch with `-LabLogServerHost 127.0.0.1`; on a phone, Sync → Server (ADR-021)

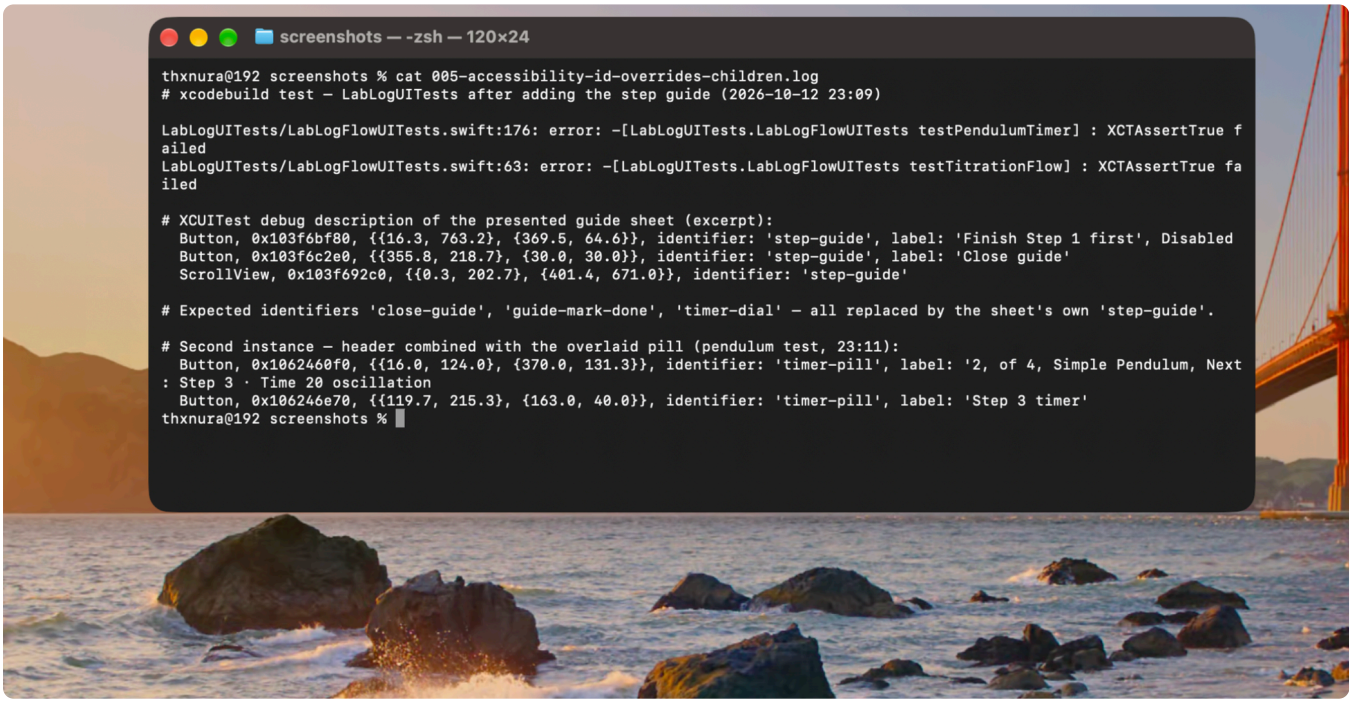
#	Date	Failure (exact error)	Cause	Fix
016	16 Oct	Full suite: titration `XCTAssertTrue failed` (Submit disabled) and sign-in `XCTAssertTrue failed` (no sheet)	A step circle tapped mid-layout never completed; one dropped tap	Every step is confirmed (`completeStep`); the sign-in tap retries once



Failure 001 — the first compile of the generated Xcode project.



Failure 002 — the crash report and test output for the SwiftData container lifetime bug.



Failure 005 — the debug description shows the Close button carrying its container's identifier.

Each error image is a Terminal window showing the log saved when the failure happened (excerpts of the `xcodebuild` output and crash reports, with my notes in `#` lines). The `.log` files sit beside the images in the repository.

9.3 AI use and prompts

I used **Claude Code (Claude Opus 5.5)** as a pair programmer. I set the direction and requirements, reviewed the output, ran the app on my iPhone, and reported problems back. The AI generated much of the code, the tests and the documentation, and diagnosed failures. Every prompt is listed exactly as I typed it:

#	Exact prompt	What it produced	Result
1	"check the status of this project and install the app in my iphone"	Status review, the Xcode project, signing, and installation on my iPhone	Partially (failure 001)
2	"it shows scaffold ready, wire lab view and other stuff, build the full app, and if u detect errors, those should be screenshoted and make a doc"	The full app (store, sync, auth, screens, camera/GPS wiring, tests) and a build log	Partially (failure 002)
3	"i get this error repetatively" (with a screenshot of the crash dialog)	A diagnosis from the crash report and the fix	Worked
4	"its look too much basic, make ui ux more like other porffesional app design with apple design skills"	The design system and a redesign of every screen	Partially (failures 003, 004)
5	"yes and u shoeud authorize as me, do not include u"	Commits under my own identity	Worked
6	"add more interactibe animations like when timer goes show them in dynamic island and show interactive timer, and when we click step add step guide as well"	The Live Activity, timer dial, step guides and deep links	Partially (failures 005, 006)
7	"yes rewrite that commits and remove all clade coauth and add my photo and name"	A history rewrite with authorship preserved	Worked
8	"this submit dialog look ugly, fix it, also make dynamic island more good with details and make proffesional"	The review sheet and a richer Live Activity	Worked (fixed failure 007)
9	"yes continue and do next build"	The local Supabase stack, end-to-end tests, lecturer role, ownership checks, and real sync from the app	Partially (failures 008, 009)
10	"what more we have to build"	A gap analysis against this brief	Worked
11	"ok do next ones"	This report draft, its diagrams and figures, demo notes, and the in-app lecturer review; checking the figures exposed failure 010	Partially (failures 010, 011); report is a first draft for my review
12	"check what we more have to build"	A second gap check: submission tasks, the risk of a LAN-only backend at a demo, optional small builds	Worked
13	"ask me the points should be filled in Appendix D"	Questions on batch mates, assets and libraries; Appendix D filled from my answers	Worked

#	Exact prompt	What it produced	Result
14	"termiansl erros all looks like styles neh, for this lets do this, make small mac app, not inside this repo. make it inside documemnt/thanura and when we fiving logs, inte rendered same to mac terminal and we can capture ss from it", then "look its huge different", "now this is too mcuh perfect, adding shadow and all…", "i said u to take ss like this" (with my own screenshot)	TermShot, a small Mac app that shows a saved log in a real Terminal window on my desktop and saves the screenshot; the error images were re-taken with it in the style of my own screenshot	Partially (window title, margin marks, inactive windows, a captured keystroke (failure 013) and two style rounds, all fixed)
15	"make word document"	`LabLog–Report.docx`, generated from the same Markdown (headings, tables, code, 14 images, page numbers)	Worked after two conversion bugs were fixed (lost spaces between links, a skipped image)
16	"so whats next we have to build"	A plan: a runtime server address first (the demo network risk), then discard draft, photo retake and step notes	Worked
17	"ok do next"	The four features above, across app and API, with 10 unit tests, 4 backend tests and a UI test; ADR-021 to 023	Partially (failure 012 caught by the new UI test and fixed)
18	"so whats next"	A short plan: update my demo notes, check dark mode, then stop adding features before the deadline	Worked
19	"so update md and do the checks"	My demo notes updated for the current app (a `Codable` trap caught and verified along the way); UI tests run in dark mode, 29 screens reviewed, Figure 13	Worked (no fixes needed)

My own role: I ran the builds on my iPhone 17 Pro and reported problems that the automated tests missed (the repeated crash dialog, the misplaced submit popover).

Lesson learned: the AI's first attempt was often close but not right, for example a layout fix that hid the badge text and UI tests with timing races. Insisting on running real builds, real tests and screenshots after every change is what made the result reliable.

10. Testing and verification

Suite	Count	What it proves
`StepConstraintEngineTests`	12	Precedence, guard order, timer anchoring, messages, reachability, Kahn order, cycles, dangling/self edges, bundled catalog validity
`SwiftDataStoreTests`	15	Outbox rules, revision guard, duplicate completions, DTO contents, submit once, catalog replace, tombstones, photo removal, step notes, submitted records locked

Suite	Count	What it proves
`SyncEngineTests`	8	Draining, photo-first ordering, per-item failure, network error, offline/signed-out, no backend, cyclic catalog skipped, a synced tombstone leaving the device
`ServerEndpointTests`	3	Host replacement keeps scheme, ports and path; switching and resetting; host validation
`LiveActivityAndGuideTests`	9	Which step drives the timer, deep-link parsing, guides on every step, catalog upgrade, server JSON decoding, lecturer role read from the JWT (and not from user-editable metadata)
UI tests (XCUITest)	5	The full titration flow to submission (GPS stamp checked); timer, dial and Dynamic Island; sign-in and sync to the real backend; a student submits and a lecturer reviews it; retake, note, discard and the server-address check. Titration, timer and editing also run in dark appearance
Backend end-to-end (`node --test`)	14	Auth (401), catalog and guides, idempotent sync, 422 validation, anti-hijack 403/409, signed upload retry, lecturer 403/200 with working photo URLs, storage URLs following the caller's host, retake replacing the stored photo, draft tombstones (idempotent, drafts only, owner only)

All 47 unit tests, 5 UI tests and 14 backend end-to-end tests passed on 16 October 2026 (Xcode 27.0, iOS 26 simulator; Node 22; Supabase local stack with Postgres 17).

11. Limitations and future work

- **Hosting.** The backend runs as a local Supabase stack plus the Express API on a laptop; a hosted deployment only needs the same SQL and new configuration (ADR-018). Because the server address can be changed in the app, the laptop can serve any network, including a phone hotspot (ADR-021). Finding the address is still manual; Bonjour discovery would automate it.
- **Conflicts.** Records have a single author, so the server applies the last write. Shared or group records would need a merge strategy.
- **Live Activity "ready" state.** It relies on the activity's stale date; push-token updates would make the switch instant.
- **Lecturer experience.** Lecturers can review submissions in the app, but grading (comments, marks) is future work. The review screens need a connection.
- **Platforms.** iPhone only; iPad layouts and Android are out of scope.

Appendix

A. Running the project

```
# Backend (needs Docker + Supabase CLI)
cd lablog-backend
supabase start -x realtime,imgproxy,mailpit,postgres-meta,studio,\
                  edge-runtime,logflare,vector,supavisor
npm install
npm run local:setup -- --host <Mac LAN IP>  # .env, demo users, iOS config
npm run dev                                 # API on :3100
npm run test:e2e                            # 14 end-to-end tests

# iOS: paste the printed values into Config/Secrets.xcconfig, then
xcodebuild test -project LabLog.xcodeproj -scheme LabLog \
  -destination 'platform=iOS Simulator,name=iPhone 17 Pro'
```

Demo accounts (local stack only): `student@lablog.test` / `LabLog-Student-1`, `lecturer@lablog.test` / `LabLog-Lecturer-1`. Without `Secrets.xcconfig` the app runs fully offline in on-device mode.

B. Environment

Xcode 27.0 · Swift 6.4 · deployment target iOS 17 · tested on an iPhone 17 Pro (iOS 27.2) and the iPhone 17 Pro simulator (iOS 26.0.1) · Node 22 · Express 4 · zod 3 · Supabase CLI 2.53 · Postgres 17.

C. Size

About 6,800 lines of Swift in the app and widget, 1,100 lines of Swift tests, 900 lines of TypeScript (API and scripts), and 550 lines of SQL and end-to-end tests.

D. Academic integrity

All images, icons and diagrams are original: drawn in code or captured from the app. No existing codebase was copied. Libraries and frameworks are used only as libraries; the custom components and the constraint logic are my own work.

Area	Libraries, frameworks and tools used
iOS app (Apple frameworks)	SwiftUI, UIKit (camera picker), SwiftData, Observation, CoreLocation, MapKit, Network (`NWPathMonitor`), Security (Keychain), UserNotifications, OSLog
Widget extension	WidgetKit, ActivityKit (Live Activity / Dynamic Island), AppIntents
iOS tests	XCTest, XCUITest
Backend runtime	Express, zod, @supabase/supabase-js, cors, dotenv
Backend tooling	TypeScript, tsx, Node 22 test runner
Infrastructure	Supabase (Postgres, Auth, Storage) via the Supabase CLI, Docker

Help to or from batch mates: None. No help was given to or received from batch mates.